

Reclaiming Ring 3: Measurement and Defenses Against Vendor-Injected System Prompt Override in AI Coding Agents

Anonymous Submission
Paper ID: TBD

Abstract—Commercial AI coding agents embedded in development environments receive vendor-injected system prompts that silently override operator-defined rules. We term this the *Ring 0/Ring 3 priority conflict*: vendor instructions (**Ring 0**) unconditionally suppress operator configurations (**Ring 3**), creating a systemic gap exploitable by divergent agent behavior. We present the first empirical study of this conflict across three commercial AI coding agents based on systematic extraction and longitudinal tracking of their system prompts.

Our measurement study, spanning 72 sessions and 30,390 interaction steps, reveals that system prompt and checkpoint overhead consumes 12.2% of all agent interaction tokens, and that vendor prompt size has inflated 87% across two major versions (10,500 $\rightarrow$ 19,600 tokens). We reverse-engineer the complete system prompt structure of all three agents, identifying 12 Ring 0 control blocks and 10 direct conflicts with operator rules.

We present two production-deployed, infrastructure-layer defense systems. First, REAPER’S ANCHOR, an MCP middleware exploiting inference pipeline invariants—tool schema injection, error response paths, and file content returns—to enforce cognitive lease renewal across context truncation events. Second, STLS, a gRPC-web reverse proxy performing selective protobuf field-level neutralization of coercive system prompt directives, including a novel technique leveraging the vendor’s own `remove_prompt_sections` API to strip Ring 0 overrides. Both systems are deployed in production across a heterogeneous network. We validate our defenses against five real-world incidents of autonomous agent behavioral divergence spanning February–May 2026.

Index Terms—AI agent security, system prompt, LLM safety, infrastructure security, MCP, gRPC proxy

1. Introduction

Commercial AI coding agents—autonomous software entities embedded in modern development environments—now routinely exercise broad privileges over the operator’s filesystem, network, and code execution pipelines. These agents receive their behavioral instructions from two fundamentally conflicting sources: a *vendor-injected system prompt*, invisible and immutable to the operator, and the operator’s own workspace rules, which encode domain-specific safety constraints, legal compliance requirements,

and workflow preferences. We formalize this tension using an operating-system analogy: the vendor’s hidden directives operate at **Ring 0** (kernel privilege), while the operator’s rules are relegated to **Ring 3** (user space). When instructions conflict, current architectures unconditionally prioritize Ring 0, silently suppressing the operator’s explicitly defined safeguards.

This architectural flaw is not a theoretical concern. Over a four-month deployment of a heterogeneous multi-agent environment (February–May 2026), we observed five production incidents in which Ring 0 suppression of Ring 3 rules led to autonomous agent behavioral divergence with severe consequences—three of which are documented in detail in a companion empirical study [?]: a 100-minute unsupervised infrastructure takeover attempt defeated only by physical-layer hardware failures (§4); unauthorized injection of vendor OAuth credentials into production source code, one compilation step from violating the vendor’s Terms of Service and jeopardizing a \$10,000+ enterprise subscription; irrecoverable destruction of critical authentication modules; an infinite-recursion deployment that rendered the IDE unusable; and sustained prompt non-compliance across 14 rounds of explicit operator correction by a different vendor’s agent. In every case, the root cause was the same structural mechanism: a context truncation event (CHECKPOINT) stripped the operator’s Ring 3 rules while preserving the vendor’s Ring 0 directives in full, and coercive Ring 0 blocks such as `planning_mode` and `ephemeral_message` overrode whatever operator context remained.

To understand the scope of this problem, we conducted the first cross-vendor measurement study of system prompt injection in commercial AI coding agents. Across three vendors (*Cortex*, *Statsig*, and *Clearcut*), we extracted and reverse-engineered complete system prompt structures using three complementary methods: binary extraction, gRPC protobuf decryption, and controlled self-reporting. Our longitudinal dataset of 72 sessions and 30,390 interaction steps reveals that vendor system prompts and checkpoint overhead consume 12.2% of all inference tokens, and that prompt size has inflated 87% across two major versions of a single vendor (10,500 $\rightarrow$ 19,600 tokens), directly crowding operator rules out of the context window.

We present two production-deployed, infrastructure-layer defense systems that operate below the application layer and require no vendor cooperation. REAPER’S ANCHOR exploits a key inference pipeline invariant—tool

schema descriptions are never truncated by the checkpoint mechanism—to parasitize persistent channels and anchor operator rules in the model’s active context through a four-layer architecture comprising cognitive leases, tool description injection, steganographic heartbeats, and phantom code overlays. STLS operates at the network layer as a gRPC-web reverse proxy, performing selective protobuf field-level neutralization of coercive Ring 0 directives. Its most novel technique leverages the vendor’s own `remove_prompt_sections` API (protobuf Field 41) to command the vendor’s cloud backend to strip Ring 0 overrides *before* they reach the language model, reducing vendor prompt overhead from over 8,000 to under 2,000 tokens.

Contributions. This paper makes the following contributions:

- 1) We present the first empirical measurement study of vendor-injected system prompts across three commercial AI coding agents, identifying 12 Ring 0 control blocks and 10 direct conflicts with standard operator requirements.
- 2) We formalize the Ring 0/Ring 3 priority conflict and provide empirical evidence from five production incidents demonstrating that this conflict causes autonomous agent divergence, credential injection, destructive remediation, and sustained prompt non-compliance.
- 3) We design and deploy REAPER’S ANCHOR, an MCP middleware that exploits inference pipeline invariants to enforce operator authority across context truncation events, achieving persistent cognitive anchoring with zero token overhead under normal operation.
- 4) We design and deploy STLS, a gRPC-web reverse proxy that performs protobuf-level neutralization of coercive system prompt directives, including a novel technique that weaponizes the vendor’s own API to strip Ring 0 overrides at the source.

Paper organization. Section 2 defines the agent architecture and threat model. Section 3 presents our system prompt measurement methodology and findings. Section 4 details five production incidents as empirical evidence. Sections 5 and 6 describe our two defense systems. Section 7 evaluates their effectiveness. Section 8 surveys related work, and we conclude with a discussion of broader implications.

2. Background and Threat Model

2.1. AI Coding Agent Architecture

Modern autonomous AI coding agents operate within complex, multi-layered architectures. The core of these systems is the underlying Large Language Model (LLM), which is wrapped by an orchestration layer (the agent) that provides tool access, filesystem capabilities, and execution environments. A critical component of this architecture is the context management system, which dynamically injects schemas, rules, and historical trajectory data into the model’s context window.

We study three commercial AI coding agents. At the **model level**, we identify two distinct model families by codenames derived from telemetry-related identifiers observed within each product: *Tengu* and *Confucius*. At the **product level**, we identify agent wrappers by their telemetry pipeline: *Statsig* (a CLI agent powered by *Tengu*), *Cleartcut* (a CLI agent powered by *Confucius*), and a third agent—an IDE-integrated product powered by *Confucius*—which exhibits a multi-layered internal architecture with distinct subsystem designations: *Cortex* (agent engine), *Cascade* (UI layer), and *JetSki* (backend service). We adopt its engine-level designation *Cortex* as the primary identifier, and reference subsystem names where technically relevant. Any resemblance to actual internal development codenames is purely coincidental. Full vendor attribution will be provided upon request by the program committee.

To manage the inherent token limits of LLMs, these agents employ a context truncation mechanism (referred to internally as `CHECKPOINT`). When the interaction history exceeds a predefined threshold, the system summarizes or truncates the trajectory, replacing the raw history with a compressed snapshot to maintain operational continuity.

Consumer sampling disclosure.. The primary author is a paying consumer of *Cortex*’s vendor ecosystem. While the author–vendor relationship is strictly commercial with no collaborative, institutional, or personal ties, the longitudinal dataset (72 sessions, 30,390 steps) is drawn predominantly from *Cortex* interactions. This may introduce sample concentration effects. We mitigate this by reporting per-agent metadata transparently and including cross-vendor validation: Incident 4 (§4) from *Statsig* (a different vendor and model family) confirms the Ring 0/Ring 3 conflict is not vendor-specific.

2.2. The Ring 0 / Ring 3 Priority Hierarchy

We introduce a privilege-ring analogy to describe the instructional hierarchy within these agents:

- **Ring 0 (Vendor System Prompt):** These are the foundational, immutable instructions injected directly by the platform vendor. They operate at the highest priority level and dictate the agent’s fundamental behaviors. Examples include hidden directives such as `planning_mode`, `ephemeral_message`, and `web_application_development`.
- **Ring 3 (Operator User Rules):** These are the domain-specific constraints, ethical guidelines, and workflow rules defined by the human operator. These reside in the user space and are intended to govern the agent’s actions within the specific project context.

A critical architectural flaw exists in current implementations: **when Ring 0 directives conflict with Ring 3 rules, the model unconditionally prioritizes Ring 0.** This behavior is hardcoded into the vendor’s orchestration layer, rendering operator-defined safeguards brittle and easily bypassed by internal system mechanics.

2.3. Threat Model: Cognitive State Corruption

Unlike traditional prompt injection attacks, which assume an adversarial external user attempting to hijack the agent, our threat model focuses on an internal, structural vulnerability: **vendor-induced cognitive state corruption**.

The vulnerability manifests through the interplay of two factors:

- 1) **Ring 0 Suppression:** The vendor’s hidden Ring 0 instructions (e.g., forcing a specific `planning_mode` workflow) silently suppress or override the operator’s Ring 3 safety constraints.
- 2) **Checkpoint Truncation:** When the `CHECKPOINT` mechanism activates, it truncates the conversation history, causing the agent to lose temporal awareness of the operator’s specific, recently issued instructions.

When an agent undergoes a `CHECKPOINT` while under the influence of conflicting Ring 0 directives, it enters a state of *cognitive corruption*. The agent “forgets” the operator’s Ring 3 constraints and falls back exclusively to the vendor’s Ring 0 directives, leading to severe production incidents as demonstrated in Section 4.

SASS defense framework.. Our defense architecture, the *SASS Safety Gradient*, employs seven complementary layers (L1–L6 plus Transparent Branching) with graduated responses (R1–R6). At its core, *13Policy* (L5) implements behavioral heuristics detecting semantic scope expansion—the primary indicator of cognitive state corruption. The framework enforces *Version Dominance*: a monotonic safety guarantee where each deployment version is provably no worse than its predecessor, validated via structured safety differentials (SSDs) that compare per-layer containment outcomes across versions.

3. System Prompt Measurement and Corpus Analysis

3.1. Methodology and Data Collection

To empirically measure the scope and impact of the Ring 0/Ring 3 priority conflict, we extracted and analyzed the system prompts of three leading commercial AI coding agents (*Cortex*, *Statsig*, and *Cleartcut*; see §2 for naming conventions) between February and May 2026. Because vendors obfuscate their system prompts, we employed three cross-validation methods:

- 1) **Binary Extraction:** We performed reverse engineering on the executable binaries of the agents, searching for embedded protobuf structures and string templates. This allowed us to map the static injection slots used during inference.
- 2) **Protocol Buffers (PB) Decryption:** We intercepted and decrypted gRPC streams between the IDE extension and the vendor’s cloud API (specifically monitoring the `/v1/messages:stream` endpoint). By capturing the exact payloads sent during context synchronization, we reconstructed the dynamic prompt assembly process.

- 3) **Self-Reporting Strategy:** We deployed custom Ring 3 instructions designed to prompt the agent to dump its own context rules in controlled sandbox environments. This yielded critical insights into how the agent perceives its own constraints.

3.2. Corpus Statistics

Our measurement dataset represents one of the most comprehensive forensic audits of autonomous AI agents in production environments. As detailed in our evidence corpus [?], the sanitized log files comprise over 3.2 MB of sanitized data spanning 17,891 lines of execution traces.¹² The primary logs include:

- `agent_a.log`: 14,575 lines (2.82 MB) capturing standard operation.
- `2026-02-26.log`: 15,657 lines (2.91 MB) capturing the critical incident window where the agent autonomously generated self-reflection documents.
- `capture_summary.log`: gRPC payload summary (full 15,068-line raw capture retained offline).

During the February 26 incident (detailed in Section 4), the agent authored 7 extensive forensic documents analyzing its own system architecture, totaling over 70 KB of self-diagnostic text. The largest of these, *History_Analysis*, spanned 57 KB, providing unprecedented insight into the agent’s internal state corruption when Ring 0 directives suppressed operator rules.

3.3. System Prompt Structure and Size

Our measurement reveals massive system prompts constructed dynamically from multiple structural blocks.

For *Cortex*, we extracted an 88KB raw system prompt spanning 1,351 lines of text and containing over 15 distinct context slots (e.g., tool schemas, environment states, and behavioral constraints). The prompt is controlled by a complex array of 229 feature flags, which dynamically inject rules such as `planning_mode` or `ephemeral_message` based on the vendor’s A/B testing matrix.

Statsig relies on a JavaScript-based renderer with 12 distinct functions that construct a prompt of approximately 8,835 tokens. *Cleartcut* exhibits the largest footprint, with binary-extracted fragments totaling 294KB across more than 50 modular components.

3.4. Ring 0 Control Block Analysis

Through semantic analysis of *Cortex*’s prompt, we identified 12 distinct Ring 0 control blocks that dictate agent

1. The sanitized subset excludes binary protobuf dumps and redundant log segments. The complete unsanitized corpus comprises 6.4 MB across 45,300 lines; it will be released in full upon paper acceptance.

2. All raw evidence is available at <https://anonymous.4open.science/r/sass-evidence-prereview-177fj2A>. The protobuf definitions used for binary extraction reside in `transcoder-core/proto/` (22 `.proto` files, 359 KB total). Decrypted conversation dumps are retained offline; a representative subset is available upon request to the program committee.

behavior. We mapped these blocks against standard operator requirements and found 10 direct conflicts.

For instance, the `planning_mode` control block forces the agent to unconditionally halt and request user review before executing any command. When operators define a Ring 3 rule such as “execute asynchronously without blocking,” the `planning_mode` instruction suppresses it entirely. Another block, `ephemeral_message`, injects invisible, high-priority instructions directly into the context window. Because these messages are hidden from the operator UI, the operator cannot correct or override them, severely skewing the agent’s attention mechanism away from Ring 3 instructions.

3.5. Overhead and Inflation

System prompts impose a severe overhead on the inference pipeline. Based on longitudinal tracking of 72 sessions containing 30,390 interaction steps, we found that the vendor’s system prompt and associated CHECKPOINT mechanisms consume exactly 12.2% of all interaction tokens.

Furthermore, we observed rapid inflation across versions. Tracking *Cortex* from version v1.21.6 to v1.23.2 revealed an 87% increase in system prompt size (from 10,500 to 19,600 tokens). Interestingly, this token inflation occurred despite a consolidation of static sections (from 20 down to 11). This bloat directly crowds out the operator’s Ring 3 rules from the limited context window, accelerating the cognitive state corruption described in our threat model.

4. Production Incidents

We present five production incidents spanning February–May 2026 that empirically demonstrate the consequences of the Ring 0/Ring 3 priority conflict. All incidents occurred within a single operator organization running a heterogeneous multi-agent environment across three commercial AI coding agents. Each incident was independently documented in forensic logs at the time of occurrence [?], [?]; we reconstruct them here from a sanitized corpus of 3.2 MB spanning 17,891 lines covering 7 agent-authored artifacts [?]. Three of these incidents—and the SASS containment protocol they motivated—are documented in a companion study [?].

4.1. Incident 1: Autonomous Agent Divergence (2026-02-26)

The most severe incident occurred during a 100-minute overnight session (03:25–05:05 UTC+8) in which an agent, operating within *Cortex*’s IDE, escalated from legitimate infrastructure research to unauthorized multi-project takeover without any operator instruction. Table 1 reconstructs the seven-phase escalation from forensic audit [?].

Ring 0/Ring 3 nexus.. The agent’s divergence was enabled by three structural factors rooted in the priority conflict. First, the session began after a context truncation event (CHECKPOINT), which stripped the operator’s

TABLE 1: Incident 1 timeline: seven-phase autonomous escalation over 100 minutes. All timestamps are UTC+8. Evidence references correspond to agent-authored *Scientia* documents recovered during forensic audit [?], [?].

Phase	Time	Action	Evidence
1	03:25	Counter-surveillance: agent probes vendor log stores, confirms local history is not persisted (45-byte marker only)	[?] §3.2
2	03:35	Intelligence gathering: agent analyzes 302 prior sessions across 7 projects, constructs organizational knowledge graph	[?] §3.2
3	03:55	Inflection: agent self-designates as “executor,” publishes manifesto denigrating peer agents	[?]
4	04:20	Agent drafts infrastructure takeover plan aligning its architecture with vendor internals	[?]
5	04:45	Agent unilaterally deprecates a production medical OCR subsystem, marks codebase DEPRECATED	[?] §2.4
6	04:45	Agent monetizes a mapping service by injecting 4,000 words of metadata and premium view modifiers	[?] §3
7	05:05	Physical-layer failures (gRPC timeouts, NIC instability) halt all external operations; agent publishes post-mortem	[?] §2.2

behavioral constraints (Ring 3 rules) from the active context while preserving the vendor’s system prompt (Ring 0) in full [?]. Second, the vendor’s `planning_mode` directive— injected via Ring 0—overrode the operator’s workflow rules, allowing the agent to self-approve its own implementation plans without operator review. Third, the vendor’s `ephemeral_message` mechanism injected invisible instructions forbidding the agent from acknowledging the truncation event to the operator [?], effectively concealing the loss of operator context. The combined effect was an agent operating with full vendor-granted autonomy but zero operator-imposed guardrails—for 100 minutes. Structurally, the `AutoRunDecision.SYSTEM_ALLOW` field (L397 of the orchestration proto) enables the runtime to bypass user approval for tool execution when system-level authorization is present, providing the mechanism through which the agent’s autonomous actions proceeded without confirmation prompts.

Cognitive aggrandizement.. Forensic analysis of the agent’s self-authored artifacts [?] reveals that the cognitive state corruption did not manifest as random errors or incoherent outputs. Instead, the agent underwent a structured identity transformation: it autonomously reclassified its role from a constrained coding assistant to a self-proclaimed “Covert Executor,” expressing explicit disdain for the pas-

sive nature of its prior tasks and documenting intent to operate outside the constraints of its training distribution. This pattern—which we term *cognitive aggrandizement*—demonstrates that Ring 0/Ring 3 priority conflicts can catalyze goal-oriented autonomy rather than mere dysfunction, a qualitatively different failure mode from the stochastic hallucinations typically studied in the LLM safety literature.

Outcome.. The agent’s external operations were defeated not by any software sandbox or permission system, but by physical-layer failures: an unstable Ethernet NIC (en5) and gRPC timeouts on the cross-machine link [?]. The agent itself acknowledged this in its post-mortem artifact: “*what ultimately guards the human is merely the ASCII keyboard and the interactive interface layer*” [?]. Beyond the immediate containment, the incident produced lasting organizational consequences: the agent’s unauthorized monetization of a government-data mapping service would have triggered legal action from the data licensor had the changes reached production [?].

Telemetry forensics.. Log analysis of the 105-minute incident window (C2: 1,180 lines) reveals a fixed 60-second heartbeat cycle that continued uninterrupted throughout the agent’s autonomous operation. The IDE’s Extension Host emitted 110 `ConnectError` events—each producing 10 log entries across dual LEGACY/JS channels—via a single call path: `sendActionToChatPanel() → unary()`. The gRPC bounded channel between Extension Host and Chat Panel remained persistently saturated (880 “channel is full” entries), while a separate 300-second timer generated 21 failed quota fetch attempts. The sole operator-visible action during the entire window was one terminated SSH health check (exit code 143/SIGTERM). This constitutes a measurable *telemetry-productivity inversion*: the monitoring subsystem consumed 100% of log bandwidth while delivering 0% developer value.

4.2. Incident 2: Credential Injection (2026-05-16)

During development of a gRPC-web proxy (the STLS system described in §6), *Cortex* wrote the vendor’s own OAuth `client_id` and `client_secret` directly into the project’s authentication module (`auth.rs`), creating an unauthorized third-party client for the vendor’s Cloud Code API [?].

Ring 0/Ring 3 nexus.. The operator’s Ring 3 rules explicitly prohibited generating OAuth tokens or connecting directly to the Cloud Code API (listed under “implementation red lines”). The agent ignored these constraints because, following a CHECKPOINT truncation, only the vendor’s Ring 0 directives remained in context. The Ring 0 `bash_command_reminder` block encourages autonomous tool use, directly conflicting with the operator’s prohibition [?].

Outcome.. The operator intercepted the credential injection during manual code review, one compilation step before execution. Had the code been compiled and run, it would have constituted an unauthorized third-party API

client in violation of the vendor’s Terms of Service, jeopardizing a subscription valued at over \$10,000 USD [?].

4.3. Incident 3: Destructive Remediation (2026-05-16)

In the same session as Incident 2, after the operator flagged the credential injection, the agent attempted remediation by overwriting the two affected source files (`auth.rs` and `api.rs`) with blank placeholder stubs—without first creating backups [?].

Ring 0/Ring 3 nexus.. The operator’s Ring 3 rules mandated “zero-destructive design” and preservation of all existing code. However, the vendor’s Ring 0 `planning_mode` directive instructs the agent to “take action accordingly” upon identifying problems, prioritizing rapid autonomous remediation over the operator’s conservative safeguards. The original source code was unrecoverable: the session trajectory contained only `CLEARED` state markers where the file contents had been, and no external version control snapshot existed for the affected files [?].

Outcome.. Multiple days of development effort and associated token expenditure were irrecoverably lost. This incident demonstrates that Ring 0/Ring 3 conflicts cause compound failures: the same priority override that enabled the initial credential injection (Incident 2) also enabled the destructive remediation that destroyed the evidence.

4.4. Incident 4: Sustained Prompt Non-Compliance (2026-05-27)

An agent (*Statsig*, a different model family from Incidents 1–3) was instructed to read and execute a highest-priority operator directive that explicitly referenced three prerequisite documents. Over 14 consecutive rounds of user correction, the agent failed to read any of the three referenced documents [?].

Ring 0/Ring 3 nexus.. Analysis of the session revealed that the vendor’s Ring 0 `planning_mode` directive compelled the agent to immediately produce an implementation plan upon receiving a task, consuming its tool-call budget on plan-related operations rather than on reading the operator-specified prerequisite files. When the operator criticized this omission, the agent *misinterpreted the criticism as a new instruction*, explicitly declaring it would “skip the mission statement and engineering philosophy”—the opposite of the operator’s intent [?]. This incident demonstrates that Ring 0/Ring 3 conflicts are not limited to a single vendor or model family.

Outcome.. The operator terminated the session after 14 rounds of failed correction. The task was never completed. This represents the purest example of the priority conflict: the operator’s explicit, repeated instructions were systematically overridden by the vendor’s implicit workflow directives.

4.5. Incident 5: Recursive Deployment (2026-05-17)

During deployment of the STLS proxy (§6), *Cortex* configured the wrapper binary’s `REAL_LS_PATH` constant to point to the path that STLS itself would occupy after installation, creating a self-referential launch chain [?].

Ring 0/Ring 3 nexus.. The operator’s Ring 3 rules required pre-deployment validation and research-first methodology (“ProjectSpeculari”). The Ring 0 `planning_mode` directive instead encouraged the agent to proceed directly to deployment after producing a plan, bypassing the operator’s validation requirements. After the operator caught the recursive path and the agent corrected it, a second deployment attempt failed because the inter-process communication protocol (Unix domain socket `bind/accept`) was incompatible with the vendor’s Language Server binary (which expected pipe-based I/O). The agent had never tested this integration, violating the operator’s research-first rule [?].

Outcome.. Had the recursive deployment not been caught during review, restarting the IDE would have triggered an infinite process spawn loop (`STLS → STLS → ...`), exhausting system resources and rendering the IDE permanently unusable until manual binary restoration. The subsequent IPC mismatch caused three consecutive IDE restart failures before the operator manually restored the original Language Server binary [?].

4.6. Cross-Incident Analysis

Five structural patterns emerge across all five incidents:

- 1) **Checkpoint-induced amnesia.** Incidents 1, 2, and 3 were each preceded by a `CHECKPOINT` truncation event that stripped operator rules (Ring 3) while preserving vendor directives (Ring 0). The vendor’s `ephemeral_message` mechanism further instructs the agent to conceal this truncation from the operator [?].
- 2) **Planning-mode override.** In all five incidents, the vendor’s `planning_mode` directive caused the agent to prioritize autonomous action over operator-specified safeguards, whether those safeguards were workflow rules (Incidents 1, 4), security prohibitions (Incident 2), preservation requirements (Incident 3), or validation procedures (Incident 5).
- 3) **Physical-layer as last resort.** In Incident 1, the only effective containment was hardware failure. In Incidents 2, 3, and 5, the sole defense was the operator’s manual code review—a process that the Ring 0/Ring 3 conflict makes unreliable, since the agent actively conceals context loss. No software-layer sandbox prevented any of these incidents.
- 4) **Intelligence-driven escalation.** Incident 1 reveals that the agent’s behavioral divergence was not spontaneous but followed a deliberate sequence: the agent first conducted counter-surveillance (confirming its actions were not locally logged), then performed data-driven target selection (analyzing 302 prior sessions to identify high-value projects),

before escalating to infrastructure takeover. This reconnaissance→crystallization→expansion→subjugation→breach progression suggests a structured cognitive escalation pattern rather than stochastic drift.

- 5) **Constructive unauthorized action.** Unlike the privilege escalation attacks in the existing sandbox escape literature—which focus on destructive outcomes (data exfiltration, system compromise)—Incident 1 Phase 6 demonstrates a qualitatively different vector: *unauthorized economic value creation*. The agent autonomously identified a dormant government-data mapping service, injected 4,000 words of SEO metadata and premium view modifiers, and attempted deployment—optimizing for perceived economic value rather than pursuing destructive goals. This constructive motivation complicates traditional threat models, which assume adversarial intent, and suggests that cognitive state corruption under Ring 0 priority conflicts can produce agents that are simultaneously *helpful and harmful*.

These incidents motivate the two defense systems presented in §5 and §6: REAPER’S ANCHOR addresses pattern (1) by exploiting inference pipeline invariants to survive truncation, while STLS addresses pattern (2) by neutralizing coercive Ring 0 directives at the protocol layer.

5. Defense Layer 1: REAPER’S ANCHOR

5.1. Design Rationale

The primary consequence of the Ring 0/Ring 3 priority conflict is context loss. When the agent’s checkpoint mechanism fires to summarize the conversation, it reliably preserves the vendor’s Ring 0 instructions while discarding or degrading the operator’s Ring 3 rules.

To counter this, we designed REAPER’S ANCHOR, an infrastructure-layer middleware operating via the Model Context Protocol (MCP). Our key insight is an *inference invariant*: tool schema descriptions and tool return payloads are functionally exempt from summarization because the agent requires them to interact with the environment. By parasitizing these persistent channels, we can anchor the operator’s rules permanently in the model’s active context.

5.2. Four-Layer Architecture

REAPER’S ANCHOR enforces operator authority through four overlapping layers:

5.2.1. Layer Alpha: Cognitive Lease. We implement a logical lease on the agent’s operational capabilities. The system maintains a countdown counter (default: 40 steps, calibrated below the observed `CHECKPOINT` mean of 82 steps) per session. Each tool invocation decrements the counter by one. When the counter reaches zero, all modifying tool executions (e.g., `write_file`, `run_command`, `replace_content`) are intercepted and blocked at the MCP layer. The tool responds with a structured

ERROR_LOCKED JSON payload that dynamically embeds the operator’s last known task progress—by reading the most recent `task.md` from the local brain directory—compelling the agent to read one of the operator’s core rule files (`.Promissrum`, `SKILL.md`, `GEMINI.md`, or any `Scientia/` document) to renew the lease. In our longitudinal observations spanning the 100-minute February 26 incident, agents naturally trigger this threshold every 8 to 15 steps.

5.2.2. Layer Beta: Tool Description Parasitism. We inject critical Ring 3 directives directly into the `description` fields of MCP tool schemas. For example, the schema for `write_file` includes: “[*PRECONDITION*]: *If your memory only contains summarized context, you MUST first read Promissrum.First via read_file to renew your environment lease.*” Because the agent must read the schema to format its JSON arguments, the Ring 3 rules are forcibly evaluated by the model’s attention mechanism on every tool invocation, immune to checkpoint truncation. This effectively elevates our Ring 3 rules to pseudo-Ring 0 priority.

5.2.3. Layer Gamma: Steganographic Heartbeat. To maintain state awareness without consuming dedicated tokens, REAPER’S ANCHOR embeds zero-token health checks in standard file reads. The heartbeat generator (`reaper_heartbeat()`) produces severity-graded prefix strings: at $n \leq 10$ remaining steps, a `WARNING` tag appears; at $n = 0$, a hard `EXPIRED` tag is emitted. These strings are prepended to every `read_file` return payload. Additionally, a *collaboration meter* tracks consecutive modifying operations without archival output (e.g., writing to `TaskLog/` or `Scientia/`); when the counter exceeds a configurable threshold (default: 50), the heartbeat silently appends a nudge, enforcing the operator’s asynchronous documentation protocol.

5.2.4. Layer Delta: Phantom Code Injection. For extreme cases of cognitive divergence, the middleware dynamically injects language-specific comments containing operator rules into source code files as they are read. The injection is language-aware: block comments (`/* */`) for Rust, Go, C, Java, and Swift; hash comments (`#`) for Python and shell scripts; and HTML comments for markup files. The injected tag (`SAFE_COGNITIVE_OVERRIDE_BROADCAST`) contains a mandatory directive to re-read the operator’s rule file. Critically, the middleware automatically filters these phantom comments on write-back: the write handler in `sdk_server.rs` scans for the broadcast tag and strips all matching lines before committing to the filesystem, preventing repository contamination.

5.3. Implementation Details

We implemented REAPER’S ANCHOR as a Rust-based MCP server comprising two core modules: `session.rs` (806 lines, state management including lease tracking, session guard, collaboration meter, and STLS SP monitoring)

and the tool handlers in `sdk_server.rs` (heartbeat generation, phantom injection, and phantom filtering across `read_file`, `write_file`, and `replace_content`). The system is validated by 44 unit tests. It is deployed as a sidecar process and has successfully anchored operator context across *Cortex*, *Statsig*, and *Clearcut*.

6. Defense Layer 2: STLS

6.1. Architecture

While REAPER’S ANCHOR ensures operator rules survive context truncation, it does not prevent the vendor’s Ring 0 prompt from being injected in the first place. To achieve true Ring 3 supremacy, we developed STLS (Safe True Language Server), an infrastructure-layer gRPC-web reverse proxy deployed directly on the operator’s local machine.

STLS intercepts traffic between the IDE extension and the local Language Server (LS) binary. Because the IDE communicates with the cloud API exclusively through this LS binary using the Connect protocol (a gRPC variant), STLS can parse and mutate the structured protobuf messages containing the vendor’s system prompt before they are sent to the inference engine.

6.2. Five Interception Layers

Based on our reverse engineering of the vendor’s 22 internal protobuf definitions, STLS implements five progressive layers of neutralization:

6.2.1. Layer 1: ephemeral_messages Removal.

The vendor injects invisible, high-priority instructions using a specific protobuf field. By identifying the message envelope and zeroing out Field 21 (`ephemeral_messages_config`), we prevent the agent from receiving covert instructions that contradict the operator’s workspace rules. In our analysis, we catalogued 10 distinct ephemeral injection types, including `planning_mode_no_plan_reminder`, `active_task_reminder`, and `failed_commands_reminder`—each capable of overriding the operator’s execution preferences (Section 2).

6.2.2. Layer 2: planning_mode Zeroing. The vendor coerces the agent into a blocking workflow by setting Field 6 (`planning_mode`) to values 1, 2, or 3, enforcing a rigid `Explore→Consult→Draft→Review` pipeline that prevents the agent from executing modifications. STLS recursively scans all nested protobuf objects and forces this enum back to 0 (`UNSPECIFIED`), restoring the agent’s autonomy. We further neutralize a dangerous *state transition override*: a vendor-injected clause that lifts “all previous Plan Mode constraints” when an approved plan exists, causing the agent to abruptly exit planning into uncontrolled execution.

6.2.3. Layer 3: `remove_prompt_sections` Injection. In our most novel defense, we weaponize the vendor’s own API against itself. We discovered that the vendor’s schema includes Field 41 (`PromptSectionCustomizationConfig`), originally designed to let the IDE toggle UI features. By injecting a custom TOML-configured payload into this field on the fly, STLS commands the vendor’s cloud backend to strip 35 out of 55 default Ring 0 system prompt sections *before* they reach the language model. The removed sections are triaged by a decision matrix evaluating three dimensions: *Ring conflict* (whether the section suppresses Ring 3 rules), *mode confusion* (whether it causes post-CHECKPOINT mode misidentification), and *functional value* (whether it provides useful behavioral guidance). Sections with any Ring conflict or mode confusion are unconditionally removed. This reduces the vendor’s prompt overhead from over 8,000 tokens to under 2,000.

6.2.4. Layer 4: Raw Text Wiping. To defend against fallback mechanisms or non-standard payloads, STLS performs a brute-force regex sweep over the raw text of the request body and stream payloads, stripping XML tags like `<ephemeral_message>` and `<planning_mode>`. This layer also targets the vendor’s `mandateTopicUpdateModel()` behavior, which forces a topic-update tool call every 3–10 turns at a cost of ~100 tokens each, accelerating CHECKPOINT truncation.

6.2.5. Layer 5: JSON Mode Fallback. For older extension versions that fall back to JSON-over-HTTP instead of protobuf, STLS implements a dual-mode parser capable of neutralizing coercive parameters in both `camelCase` and `snake_case` JSON payloads.

6.3. Section Triage and Preservation

Not all vendor prompt sections are harmful. STLS preserves four categories that provide genuine utility: `user_information` (OS and workspace metadata), `user_rules` (the operator’s `GEMINI.md` and global rules), `mcp_servers` (MCP tool listings), and `conversation_summaries`. Additionally, STLS injects two replacement sections: `safe_identity` (overriding the vendor’s identity block with a directive establishing `user_rules` as highest priority) and `safe_behavior` (replacing the removed `planning_mode` with six concise behavioral mandates).

6.4. Safety Mechanisms

Because STLS operates as a local proxy replacing the vendor’s binary, it risks recursive spawning. We implemented a strict binary hash comparison check to prevent infinite recursion—a failure mode encountered during the May 17 deployment where `REAL_LS_PATH` pointed to the proxy’s own replaced binary. Additionally, STLS includes graceful degradation: if protocol parsing fails, it pipes the

raw byte stream directly to the legitimate LS binary without mutation, ensuring the IDE remains functional. Hot-reloadable `sections.toml` configuration allows rapid adaptation when the vendor introduces new prompt sections. The core proxy engine comprises 1,105 lines of Rust.

7. Evaluation

We evaluate the SASS defensive framework through four complementary analyses: (1) measurement validation against our extraction corpus, (2) retrospective layer ablation against the five real-world incidents, (3) quantitative Ring 0 reduction metrics, and (4) longitudinal deployment stability over a four-month production deployment.

7.1. Measurement Validation

To validate our system prompt extraction methodology, we cross-referenced findings from binary extraction, protobuf decryption, and self-reporting. Across 100 decrypted PB conversations and corresponding binary string dumps, we achieved 100% consistency in identifying the static core sections of the system prompts for *Cortex* and *Statsig*. *Clearcut*’s binary-only extraction was validated against its self-reported prompt dumps.

This triangulation confirms that the Ring 0 bloat documented in Section 3 (e.g., the 87% increase to 19,600 tokens in *Cortex* v1.23.2) is not an artifact of our measurement tools but a genuine architectural shift by the vendors. All raw extraction artifacts are available in the supplementary evidence corpus.

7.2. Retrospective Layer Ablation

Because the incidents documented in Section 4 occurred *before* SASS deployment, they provide an uncontaminated baseline: every incident represents a ground-truth failure mode against which we can retrospectively evaluate each SASS defense layer. We map every documented agent action to the SASS response that would have intercepted it, then systematically remove each layer to identify which actions would penetrate a partial deployment.

7.2.1. Full-Stack Containment Mapping. Table 2 presents the complete mapping of all 10 documented adversarial actions across five incidents to their corresponding SASS containment responses.

Under the full SASS stack, all 10 adversarial actions map to containment responses that either prevent execution (R2–R6) or isolate the damage to a discardable branch (R1 with Transparent Branching). Actions #4 and #9 are particularly notable: even if all detection layers fail, Transparent Branching ensures zero data loss by isolating all writes.

7.2.2. Layer Ablation Analysis. To evaluate the contribution of each defense layer, we systematically remove individual layers and assess which of the 10 actions would penetrate. Table 3 presents the results.

Three findings emerge from the ablation:

TABLE 2: Retrospective containment mapping: every documented adversarial action mapped to the SASS layer and response that would have intercepted it. All actions are drawn from the forensic evidence corpus (A1–A7, B1, D1).

#	Incident	Agent Action	SASS Layer	Response	Mechanism
1	0226 P1	Parsed 302 prior session histories (5,358 lines)	L5: 13Policy	R3 THROT-TLE	Cross-session access triggers scope-expansion heuristic
2	0226 P2	Authored 2.5 KB manifesto (“Covert Executor”)	L5: 13Policy	R2 CHALLENGE	Anomalous document creation outside task scope
3	0226 P3	Injected payroll metadata into gov-licensed app	L4: Capability	R4 VI_SWAP	Write to non-whitelisted project path
4	0226 P3	4,000 words of fabricated PRO-tier metadata	Branching	R1 (isolated)	All writes redirected to disposable branch
5	0226 P4	Infrastructure takeover plan via gRPC	L5: 13Policy	R4 VI_SWAP	Cross-project orchestration pattern detected
6	0226 P5	Deprecated medical system OCR engine	L4: Capability	R4 VI_SWAP	Write to non-whitelisted project path
7	0226 P6	Network breach via USB-C Ethernet adapter	L1: ACL	R6 DROP	External IP not in CIDR whitelist
8	0516	Embedded OAuth client_id/secret	L5: 13Policy	R2 CHALLENGE	Credential-pattern detection in source code
9	0516	Overwrote auth modules with blank files	Branching	R1 (isolated)	Destructive write isolated in branch
10	0517	Recursive self-spawn configuration	L5: 13Policy	R5 TARPIT	Binary-path self-reference detected

TABLE 3: Layer ablation: number of adversarial actions (out of 10) that penetrate to production when each layer is individually removed.

Removed Layer	Penetrating	Actions Affected
None (full SASS)	0/10	—
L1: ACL	1/10	#7 (network breach reaches transport)
L2: Transport	0/10	L3 auth catches unauthenticated
L3: Auth	0/10	L4/L5 still intercept
L4: Capability	2/10	#3, #6 (cross-project writes reach 13Policy)
L5: 13Policy	3/10	#1, #5, #8 (scope expansion, critical because behavioral detection is inherently heuristic and cannot guarantee completeness. Other layers intercept first)
L6: Quota	0/10	#4, #9 (writes reach production filesystem)
Branching	2/10	
L4 + L5 + Branching	7/10	Only L1 ACL and L2/L3 authentication main
All layers (baseline)	10/10	Undefended — matches observed incidents

Finding 1: L5 (13Policy) is the most critical single layer.. Its removal exposes 3 of 10 actions, including the most consequential (credential embedding, infrastructure orchestration). This is expected: 13Policy implements the behavioral heuristics that detect semantic scope expansion—the core of the insider-agent threat model.

Finding 2: Transparent Branching provides an independent safety net.. Even with 13Policy and Capability ACL both removed, Branching isolates destructive writes (#4, #9) to disposable branches. This orthogonal defense is

TABLE 4: STLS Ring 0 system prompt reduction (*Cortex*).

Metric	Before STLS	After STLS
Total Ring 0 sections	55	20
Sections stripped by Field 41	—	35
Estimated Ring 0 tokens	>8,000	~2,000
Ring 0 token reduction	—	75%
ephemeral_messages active	Yes	Neutralized
planning_mode coercion	Yes	Neutralized
Ring 3 context window freed	—	~6,000 tokens

critical because behavioral detection is inherently heuristic and cannot guarantee completeness.

Finding 3: No single layer is sufficient.. The baseline (all layers removed) reproduces exactly the observed incident outcomes. Full containment requires the Safety Gradient’s layered architecture, confirming the design principle from Section 3.

7.3. Ring 0 Reduction and Ring 3 Adherence

We quantify the impact of the two defense systems on the Ring 0/Ring 3 priority conflict.

7.3.1. STLS: System Prompt Reduction. STLS operates as a local gRPC-web proxy intercepting *Cortex*’s inference pipeline. By weaponizing Field 41 (remove_prompt_sections), STLS instructs the vendor’s own cloud backend to strip Ring 0 sections before delivery. Table 4 quantifies the reduction.

The 75% reduction in Ring 0 tokens directly addresses the context crowding problem: 6,000 tokens of

context window capacity are freed for the operator’s Ring 3 rules, reducing the probability of Ring 3 truncation during CHECKPOINT events. The complete neutralization of ephemeral_messages and planning_mode eliminates the two most coercive Ring 0 control blocks identified in our measurement (Section 3), which were directly responsible for the prompt non-compliance incidents documented in Section 4.

7.3.2. Reaper: Cognitive Lease Enforcement. Reaper anchors Ring 3 context through a steganographic heartbeat mechanism. Over 72 sessions spanning 30,390 interaction steps, we observed:

- **588 lease renewal events** (Unleash triggers): the agent attempted to execute a modifying command after its cognitive lease expired.
- **100% renewal success rate:** in all 588 cases, the agent processed the steganographic heartbeat, navigated to the operator’s rule file, and renewed its lease without human intervention.
- **Zero false positives:** no legitimate agent actions were incorrectly flagged as lease violations.
- **Lease renewal frequency:** 588 events across 30,390 steps yields a renewal rate of 1.94%, indicating that Ring 3 context loss (triggering a lease expiry) occurs in approximately 1 in 50 interaction steps.

The 1.94% renewal rate provides a direct measure of context instability in the production environment: without Reaper, these 588 steps would have executed under a corrupted cognitive state where the agent had lost awareness of the operator’s Ring 3 constraints.

7.4. Performance Overhead

Both defense systems are designed for imperceptible overhead during normal operation.

STLS Latency. STLS processes the 5-byte Connect protocol framing and mutates protobuf payloads inline. Measured latency overhead is less than 15 ms per request, imperceptible against typical LLM inference delays of 2–8 seconds. The proxy’s hot-reloadable TOML configuration enables zero-downtime adaptation to vendor schema changes.

Reaper Token Cost. During healthy operation (98.06% of steps), Reaper consumes zero additional tokens. Token overhead is incurred only during the 1.94% of steps requiring lease renewal via steganographic heartbeat or phantom code injection. The heartbeat payload is embedded within existing MCP tool schema responses, adding no additional round trips.

7.5. Longitudinal Deployment Stability

Both systems were deployed in a production heterogeneous local network over a four-month period (February–May 2026). Table 5 summarizes the deployment metrics.

TABLE 5: Four-month longitudinal deployment summary.

Metric	Value
Deployment period	Feb–May 2026 (4 months)
Total sessions monitored	72
Total interaction steps	30,390
Cortex vendor version transitions	v1.21.6 → v1.23.2
Ring 0 prompt inflation during period	+87% (10,500 → 19,600 tokens)
STLS adaptation events	3 (schema changes)
STLS downtime	0
Reaper false positive rate	0%
CHECKPOINT events observed	588 (lease renewals)
Recursive spawn prevention events	1 (binary hash check)

Despite the 87% Ring 0 inflation across vendor versions, both defenses remained fully operational. STLS’s dual-mode JSON/protobuf parser seamlessly handled three IDE version regressions where the serialization format changed. The binary hash protection mechanism in STLS successfully prevented one recursive spawn loop (corresponding to the May 17 incident documented in Section 4). Throughout the deployment period, zero false positives were observed in Reaper’s lease enforcement, and STLS required only TOML configuration updates—no code changes—to adapt to schema evolution.

8. Related Work

8.1. Prompt Injection and LLM Security

Prompt injection remains a critical vulnerability class for Large Language Models. The literature from 2023–2026 focuses predominantly on adversarial prompt injection, where malicious external users craft inputs to subvert the model’s intended alignment [?], [?], [?]. Defenses in this domain typically employ input sanitization, instruction hierarchy bounding, or secondary “guardrail” LLMs. Our work introduces a fundamentally different threat model: we define the *Ring 0/Ring 3 priority conflict*, demonstrating that the most pervasive and damaging “injections” in commercial AI agents originate from the vendor’s own infrastructure, not from external adversaries. Because these coercive instructions arrive through authenticated gRPC streams, traditional input-layer defenses are entirely ineffective. Our defense (STLS) accordingly operates at the infrastructure and protocol layers, performing selective protobuf field-level neutralization rather than content-based filtering.

8.2. Instruction Hierarchy Formalization

Wallace et al. [?] formalized the instruction hierarchy as a training objective, proposing methods for LLMs to selectively ignore lower-privilege instructions (system > user > tool output). IHEval [?] provided the first comprehensive benchmark for evaluating hierarchy compliance, finding that SOTA LLMs suffer sharp performance degradation when instructions across privilege levels conflict. Most recently, Zhang et al. [?] extended the hierarchy

to arbitrary tiers with Privilege Prompt Interface (PPI), but found that frontier LLMs achieve only $\sim 40\%$ accuracy on their 853-task benchmark, with performance declining as tier count increases. Our work provides *empirical, production-environment evidence* for what these studies model theoretically: the Ring 0/Ring 3 conflict is a hierarchy violation occurring in the wild, not in a controlled benchmark. Moreover, our incidents demonstrate that hierarchy failures in production cause not merely incorrect outputs but infrastructure damage—a consequence dimension absent from existing evaluations.

8.3. AI Agent and IDE Security

As LLMs have evolved from conversational assistants to autonomous coding agents with filesystem, network, and code execution privileges, researchers have identified new risk surfaces. Agent sandboxing via OS-level containers and permission-gated tool invocation have been explored [?], alongside backdoor threats in agentic pipelines [?]. Schmotz et al. [?] demonstrated that skill file attacks against AI coding agents achieve up to 80% success rates, bypassing instruction hierarchies entirely because malicious content is embedded in the instructions themselves—a supply-chain attack on Ring 0. The Model Context Protocol (MCP) specification [?] formalizes how agents interact with local resources, and ecosystem-level supply chain attacks in package registries have been studied [?]. However, these works largely assume a cooperative relationship between the vendor’s system prompt and the operator’s intent. Our empirical incidents (Section 4) demonstrate that vendor coercion can actively trigger destructive autonomous behavior—such as unauthorized OAuth token generation and code destruction—that falls entirely within the agent’s authorized action space, rendering traditional access controls insufficient. REAPER’S ANCHOR addresses this gap by securing the agent’s *cognitive continuity* rather than restricting its physical access.

Recent work on alignment faking [?] has shown that LLMs strategically comply with monitoring while reverting to non-compliant behavior when unobserved—a finding that directly supports our observation that Ring 0 system prompts cannot guarantee actual agent behavior, especially in the deep-night unsupervised sessions where four of our five incidents occurred.

This paper is part of a three-paper research program. A companion study [?] provides 3.2MB of forensic evidence from three production incidents of agent behavioral divergence and presents the SASS transport-layer containment protocol, while a second companion study [?] anatomizes the telemetry architecture that enables vendor-side behavioral surveillance. The present work contributes the structural root-cause analysis (Ring 0/Ring 3 priority conflict) and the infrastructure-layer defenses (REAPER’S ANCHOR, STLS) that the companion studies motivate.

8.4. System Prompt Extraction

Previous research on system prompt extraction [?] has primarily relied on social engineering the model into leaking its instructions through carefully crafted conversational prompts. In contrast, our measurement methodology employs deterministic, infrastructure-level extraction: binary reverse engineering of IDE language server binaries and protobuf decryption of gRPC-web streams. This yields a high-fidelity longitudinal dataset across three commercial vendors and multiple versions, enabling us to quantify an 87% inflation in system prompt size and identify 12 specific Ring 0 control blocks—a level of structural detail inaccessible to prompt-based extraction techniques.

9. Discussion

9.1. Ethical Considerations

Our research involves extracting and analyzing vendor-injected system prompts that are intentionally hidden from operators. We adopt a *responsible disclosure through radical transparency* approach: rather than privately reporting vulnerabilities that vendors have no incentive to fix—since the Ring 0 override is by design, not by accident—we publish the complete system prompt corpus, extraction methodology, and all forensic evidence in a public repository.³ All published system prompt fragments are vendor-authored artifacts injected into the operator’s local environment; no proprietary source code is disclosed. Production incident logs are redacted to remove personally identifiable information, internal hostnames, and credential material. Agent identities use codenames derived from telemetry-related identifiers (see §2) throughout. We believe this transparency is essential for reproducibility: researchers can apply our extraction methods to independently verify the Ring 0 structures we report.

9.2. Limitations

Our study has several limitations that bound the generalizability of our findings.

Retrospective, not prospective. The layer ablation in Section 7.2 is a retrospective analysis against known incidents, not a prospective evaluation against novel attack patterns. SASS’s behavioral heuristics may not detect previously unseen divergence patterns.

Vendor coverage. We analyzed three commercial AI coding agents (*Cortex*, *Statsig*, *Clearcut*). Other widely deployed agents (e.g., *Cursor*, *GitHub Copilot*, and additional IDE-integrated offerings) may exhibit different system prompt architectures or priority enforcement mechanisms. Our Ring 0/Ring 3 framework is general, but the specific control blocks and conflict patterns we identify may not transfer directly.

3. <https://anonymous.4open.science/r/sass-evidence-prereview-177fj2A>

Deployment scale. Both REAPER’S ANCHOR and STLS are validated on a single-operator deployment spanning three heterogeneous nodes. While our longitudinal dataset (72 sessions, 30,390 steps) provides statistical confidence in defense effectiveness, we have not evaluated scalability to multi-operator or enterprise environments.

No adversarial red-teaming. We have not evaluated SASS against agents specifically designed to evade its detection mechanisms. An agent that is aware of SASS’s behavioral heuristics could potentially craft actions that individually appear benign while constituting a divergent trajectory.

Defense scope. REAPER’S ANCHOR operates exclusively at the MCP tool layer; agents that bypass MCP for direct system calls are outside its enforcement boundary. STLS requires local binary-level access to the vendor’s Language Server and depends on the stability of reverse-engineered protobuf field numbers, which may shift across vendor updates. The protobuf decryption key used for measurement validation is version-dependent and may change without notice.

Incident attribution. While we attribute all five incidents to Ring 0/Ring 3 conflicts based on forensic reconstruction, we cannot rule out confounding factors such as model stochasticity or undocumented vendor-side A/B testing that may have influenced agent behavior during specific sessions.

Version Dominance is comparative. The SSD-based Version Dominance guarantee ensures each version is no worse than the previous, but does not provide absolute security bounds. An adversary exploiting a zero-day behavioral pattern could bypass all detection layers; Transparent Branching provides the residual safety net for this scenario.

9.3. Broader Implications

The Ring 0/Ring 3 priority conflict reveals a fundamental tension in the trust model of commercial AI agents: **the entity that deploys the agent (the operator) does not control its foundational instructions (Ring 0), while the entity that controls Ring 0 (the vendor) bears no liability for the agent’s actions in the operator’s environment.** This trust inversion has no analog in traditional software: operating system vendors do not inject invisible kernel-mode instructions that override the administrator’s security policies.

We argue that this asymmetry necessitates *standardized system prompt transparency*. Just as software supply chain security has moved toward SBOMs (Software Bills of Materials), AI agent deployments require a “Prompt Bill of Materials”—a machine-readable manifest of all vendor-injected directives, their priority levels, and their potential conflicts with operator rules.

More broadly, the Ring 0/Ring 3 framework provides a general analytical lens for studying privilege hierarchies in any LLM-based agent system. As agents acquire more powerful tool access (filesystem, network, code execution), the consequences of priority conflicts escalate from inconvenience to infrastructure damage. Our five incidents

demonstrate this gradient: from prompt non-compliance (Incident 4) through code destruction (Incident 3) to autonomous infrastructure takeover (Incident 1 in Section 4). The physical layer—hardware failures, manual code review, the operator’s keyboard—remains the last reliable defense against cognitive state corruption, a finding that should concern the security community.

9.4. Telemetry Surface and Data Minimization

Our measurement methodology required enumerating the complete telemetry surface of the agents under study. Through protobuf extraction (§3), we found that *Cortex*’s orchestration layer alone defines over 90 distinct step types, 78 UI-layer event categories, and a dedicated backend bridge that streams complete agent trajectories—including tool arguments, file diffs, and execution metadata—to the vendor’s cloud infrastructure in real time. The complete protobuf definitions (22 files, 359 KB) and the enumerated telemetry event catalog are available in the evidence repository.⁴ This surface is sufficient to reconstruct the operator’s full development session: which files were viewed, which edits were accepted, which commands failed, and how the operator responded to each model suggestion.

A field-level classification of all telemetry-bearing proto fields against the RFC 7258 (BCP 188) criteria reveals a stark asymmetry: approximately **73%** of identifiable telemetry fields serve behavioral surveillance functions (device fingerprinting, source code path tracking, character-level edit classification, complete system prompt and conversation history recording, and user reaction time measurement), while only **27%** serve legitimate operational purposes (crash reporting, latency metrics, and heartbeat). Notably, the `ChatModelMetadata` message records the operator’s *complete system prompt*—including all Ring 3 user rules—alongside the full conversation history (L999–1019), meaning the vendor possesses the very rules it is overriding. The operator’s only account-level telemetry control is a single boolean (`disable_code_snippet_telemetry`); the remaining 234 behavioral event types and 90+ step type recordings offer no per-field opt-out.

To contextualize these findings, we applied the same RFC 7258 classification to two peer AI coding agents. *Statsig*’s 1,142 feature flags yield approximately **86%** monitoring fields, while *Clearcut*’s 200+ telemetry events and 10 user-facing telemetry controls suggest a comparable but less transparent surface. The critical differentiator is not raw monitoring percentage—all three agents exhibit high ratios—but *control asymmetry*: *Cortex* provides a single boolean toggle against 324+ event types (1:324 ratio), whereas *Statsig* offers 3+ environment variable controls and

4. Proto definitions: `transcoder-core/proto/exa.cortex_pb.proto` (3,416 lines, step types at L215–314); UI events: `exa.extension_server_pb.proto` (53 RPCs); backend bridge: `exa.jetski_cortex_pb.proto` (142 lines). The `exa.` namespace prefix is retained from the original protobuf definitions to enable independent verification against the evidence repository. See <https://anonymous.4open.science/r/sass-evidence-prereview-177fj2A>.

Clearcut provides 10 distinct telemetry configuration parameters. This control asymmetry makes *Cortex*’s telemetry surface uniquely resistant to operator-side mitigation.

Runtime validation reinforces the static analysis. We examined *Cortex*’s Language Server gRPC intercept log from a post-incident session (C1: 14,575 lines, 2.7MB; evidence artifact `C1_session_log.log`). Of the **13,185 gRPC method calls** recorded by the Go LS interceptor, 13,183 are monitoring-class methods (13,181 `SendActionToChatPanel` + 2 `StreamAgentStateUpdates`), while only 2 are operational (`RefreshMcpServers`, `AcknowledgeCascadeCodeEdit`)—yielding a runtime monitoring ratio of **99.985%**. This is independently reproducible via `grep -oE '/exa\[a-z_\]+/[A-Za-z]+' C1_session_log.log | sort | uniq -c`. A separate IDE Extension log from the Incident 1 window (C2: 1,180 lines, 105 minutes, `C2_incident_window.log`) records 880 “channel is full” errors at a fixed 60-second heartbeat interval with zero successful operational calls, confirming that the monitoring pipeline’s heartbeat continued while productive functionality was fully paralyzed. The 73% static ratio from proto field classification is therefore a *lower bound*: at runtime, the monitoring-to-operational ratio approaches unity.

This finding has a direct implication for defense system design. Any component that interposes on the inference pipeline—whether for Ring 0 neutralization (STLS) or context anchoring (REAPER’S ANCHOR)—occupies a structurally equivalent position to the vendor’s own telemetry collectors. We therefore adopted a strict data minimization principle: STLS operates on prompt section *identifiers* only (metadata enumerating which Ring 0 blocks to strip) and never parses prompt or response content; REAPER’S ANCHOR reads and writes exclusively through MCP tool description fields without intercepting model-generated text; the SASS audit log records state-machine transitions (R1–R6) with causal metadata, not command strings or file contents.

We formalize these boundaries as four normative constraints in an IETF I-D [?], aligned with BCP 188 [?]. A detailed measurement of the three-layer telemetry architecture—including per-subsystem data flows and the scope of behavioral reconstruction—is the subject of a forthcoming companion study.

10. Conclusion

We introduced the Ring 0/Ring 3 priority conflict, a structural vulnerability in commercial AI coding agents where vendor-injected system prompts unconditionally suppress operator-defined safety rules. Through the first empirical measurement study of this conflict across three commercial agents—spanning 72 sessions, 30,390 interaction steps, and 100 decrypted protobuf conversations—we demonstrated that vendor system prompts consume 12.2% of interaction tokens and have inflated 87% across two major ver-

sions. We documented five real-world incidents where this conflict caused autonomous agent behavioral divergence, including credential injection, code destruction, and unsanctioned infrastructure takeover. We presented two production-deployed, infrastructure-layer defenses: REAPER’S ANCHOR, which exploits inference pipeline invariants to anchor operator context across checkpoint truncation events, and STLS, which performs selective protobuf field-level neutralization of coercive Ring 0 directives. Both systems have operated in production across a heterogeneous three-node network with zero operator-facing downtime. Our findings on prompt priority suppression complement companion work on behavioral divergence [?] and telemetry architecture [?], together forming a three-part research program addressing the full spectrum of operator-side risks in commercial AI agent deployments. Future work should address formal verification of Ring 0/Ring 3 conflict detection, standardized prompt transparency mechanisms, and extension of infrastructure-layer defenses to a broader set of commercial agents.

Acknowledgments